SÉCURITÉ

Travaux pratiques : contrôle d'accès basé sur les rôles (RBAC) avec Oracle Database 19c

0 Préparation de l'environnement

1 Utilisateurs

2 Privilèges

3 Rôles et modèle RBAC

4 Profils

5 Audit unifié et FGA

6 Étude de cas : Banque Atlas



0 Sommaire

0 Préparation de l'environnement 4
0.1 Script d'initialisation 5

1 Utilisateurs 7
1.1 Créer et authentifier 8
1.2 Tablespaces et quotas 8
1.3 États d'un compte 9
1.4 Utilisateurs communs et locaux 9
1.5 Exercices 9

2 Privilèges 12
2.1 Privilèges système 13
2.2 Privilèges objet 13
2.3 Transmettre un privilège 14
2.4 Exercices 14

3 Rôles et modèle RBAC 16
3.1 Créer et attribuer des rôles 17
3.2 Quand une modification prend-elle effet ? 17
3.3 Rôles prédéfinis 18
3.4 Exercices 18

4 Profils 21
4.1 Limites de ressources 22
4.2 Politique de mots de passe 22
4.3 Exercices 23

5 Audit unifié et FGA 25
5.1 Ce qu'une politique peut tracer 26
5.2 Audit fin (FGA) 26
5.3 Lire, protéger et purger la trace 26
5.4 Exercices 27

6 Étude de cas : Banque Atlas 29
6.1 Le contexte 29
6.2 Les acteurs et leurs besoins 31
6.3 Travail demandé 32
6.4 Questions de synthèse 33
6.5 Barème indicatif 34

A Annexes 35
A.1 Aide-mémoire : vues du dictionnaire 35
A.2 Aide-mémoire : codes d'erreur du TP 36
A.3 Script de nettoyage 36
A.4 Références 37

0 Préparation de l'environnement

Dans ce TP, vous allez mettre en place, pas à pas, une politique de sécurité complète fondée sur le modèle **RBAC** (*Role-Based Access Control*) : qui peut se connecter, ce que chacun peut faire, dans quelles limites, et comment on le prouve. Chaque partie se termine par des exercices directs ; une étude de cas finale regroupe l'ensemble.

- ➤ ❶ **Créer et gérer des utilisateurs**
- ➤ ❷ **Accorder et retirer des privilèges**
- ➤ ❸ **Concevoir une hiérarchie de rôles**
- ➤ ❹ **Encadrer les comptes avec des profils**
- ➤ ❺ **Tracer les accès avec l'audit**

● Le saviez-vous ?

Le RBAC repose sur une idée simple : on n'accorde jamais un droit directement à une personne, mais à une **fonction** (un rôle). Quand Lina change de poste, on lui retire un rôle et on lui en donne un autre : aucun droit oublié.

Atelier 0.1 — Conditions de travail

❶ Une base Oracle Database **19c** en architecture multitenant : la CDB `ORCL` et la PDB `ORCLPDB` (noms par défaut de DBCA, à adapter).

❷ Un alias réseau `ORCLPDB` dans le fichier `tnsnames.ora` (voir le chapitre Oracle Net). Toutes les connexions du TP se font dans la PDB :

```
SQL*Plus
SQL> CONNECT sys/<mot_de_passe>@orclpdb AS SYSDBA
SQL> SHOW CON_NAME          -- doit afficher ORCLPDB
```

❸ Deux fenêtres SQL*Plus ouvertes en parallèle : l'une pour l'administrateur (`SYS`), l'autre pour l'utilisateur testé. Nous les notons **[ADMIN]** et **[USER]**.

❹ Un compte rendu dans lequel vous collez, pour chaque exercice, les commandes exécutées, les messages obtenus et vos réponses aux questions.

0.1 Script d'initialisation

Le schéma `RH` sert de terrain de jeu aux parties 1 à 5. C'est un compte *schéma seul* : il possède des tables mais personne ne peut s'y connecter directement (nous verrons pourquoi au chapitre 1).

[ADMIN] 00-init-rh.sql

```
-- Schéma applicatif sans mot de passe (schema-only account, 18c et +)
CREATE USER rh NO AUTHENTICATION
  DEFAULT TABLESPACE users QUOTA UNLIMITED ON users;

CREATE TABLE rh.departements (
  dept_id  NUMBER PRIMARY KEY,
  nom      VARCHAR2(40) NOT NULL,
  ville    VARCHAR2(40));

CREATE TABLE rh.employes (
  emp_id        NUMBER PRIMARY KEY,
  nom           VARCHAR2(40) NOT NULL,
  prenom        VARCHAR2(40),
  poste         VARCHAR2(40),
  salaire       NUMBER(9,2),
  dept_id       NUMBER REFERENCES rh.departements,
  date_embauche DATE DEFAULT SYSDATE);

INSERT INTO rh.departements VALUES (10, 'Direction',    'Rabat');
INSERT INTO rh.departements VALUES (20, 'Informatique', 'Tanger');
INSERT INTO rh.departements VALUES (30, 'Finance',      'Casablanca');
INSERT INTO rh.departements VALUES (40, 'Logistique',   'Tanger');

INSERT INTO rh.employes VALUES (1, 'Alaoui',   'Karim',  'Directeur',      45000, 10, DATE
    '2015-03-01');
INSERT INTO rh.employes VALUES (2, 'Bennani',  'Salma',  'Cheffe de projet', 28000, 20,
    DATE '2018-09-15');
INSERT INTO rh.employes VALUES (3, 'Chraibi',  'Youssef','Développeur',    16000, 20, DATE
    '2021-01-04');
INSERT INTO rh.employes VALUES (4, 'Drissi',   'Imane',  'DBA',            22000, 20, DATE
    '2019-06-10');
INSERT INTO rh.employes VALUES (5, 'El Fassi', 'Nadia',  'Comptable',      14000, 30, DATE
    '2020-02-17');
INSERT INTO rh.employes VALUES (6, 'Filali',   'Omar',   'Contrôleur',     31000, 30, DATE
    '2016-11-21');
INSERT INTO rh.employes VALUES (7, 'Ghazi',    'Hamza',  'Magasinier',      7500, 40, DATE
    '2022-05-02');
INSERT INTO rh.employes VALUES (8, 'Haddad',   'Rim',    'Responsable log.',19000, 40,
    DATE '2017-08-28');
COMMIT;

-- Vue « annuaire » : tout sauf les salaires
CREATE VIEW rh.v_annuaire AS
  SELECT emp_id, nom, prenom, poste, dept_id FROM rh.employes;
```

✎ **Attention !**

Ne travaillez jamais dans la racine `CDB$ROOT` pour ce TP. Si `SHOW CON_NAME` affiche `CDB$ROOT`, basculez avec `ALTER SESSION SET CONTAINER = orclpdb;` ou reconnectez-vous avec l'alias de la PDB.

Exercice 0.1 — *Vérifier l'environnement*

1. Exécutez le script, puis affichez le nombre de lignes de chaque table.
2. Interrogez `DBA_USERS` pour afficher `USERNAME`, `AUTHENTICATION_TYPE`, `ACCOUNT_STATUS` et `ORACLE_MAINTAINED` du compte `RH`. Que vaut `AUTHENTICATION_TYPE` ?
3. Combien d'utilisateurs de la PDB sont maintenus par Oracle (`ORACLE_MAINTAINED = 'Y'`) ? Combien ont été créés par vous ?

1 Utilisateurs

Dans cette partie, vous créez les comptes qui permettront d'entrer dans la base. Un utilisateur Oracle est à la fois une *identité* (qui se connecte) et un *schéma* (un espace qui peut contenir des objets).

- ➤ 1 **Créer un utilisateur et l'authentifier**
- ➤ 2 **Lui attribuer tablespaces et quotas**
- ➤ 3 **Gérer le cycle de vie du compte**
- ➤ 4 **Distinguer utilisateurs communs et locaux**
- ➤ 5 **Utiliser les comptes schéma seul et proxy**

● Le saviez-vous ?

Créer un utilisateur ne lui donne **aucun droit**, pas même celui de se connecter. Une porte d'entrée sans badge : c'est voulu. Tout droit doit être accordé explicitement — c'est le principe du **moindre privilège**.

Atelier 1.1 — Anatomie d'un CREATE USER

1 La commande complète, clause par clause :

```
[ADMIN]
CREATE USER lina                          -- nom (local à la PDB)
  IDENTIFIED BY "Lina#2026"               -- authentification par mot de passe
  DEFAULT TABLESPACE users                -- où vont ses objets par défaut
  TEMPORARY TABLESPACE temp               -- tris et jointures volumineux
  QUOTA 10M ON users                      -- espace qu'elle peut consommer
  PROFILE default                         -- limites (chapitre 4)
  PASSWORD EXPIRE                         -- changement imposé à la 1re connexion
  ACCOUNT UNLOCK;                         -- compte ouvert
```

2 Seules les deux premières lignes sont obligatoires ; les autres reprennent les valeurs par défaut de la base.

3 Le résultat se lit dans le dictionnaire : `DBA_USERS`, `DBA_TS_QUOTAS`.

1.1 Créer et authentifier

✻ Définition — Utilisateur et schéma

Un **utilisateur** est un compte déclaré dans la base, identifié par un nom et un mode d'authentification. Le **schéma** est l'ensemble des objets (tables, vues, procédures...) appartenant à cet utilisateur ; il porte le même nom. `rh.employes` désigne la table `EMPLOYES` du schéma `RH`.

Mode	Clause	Usage
Mot de passe	`IDENTIFIED BY mdp`	Cas général du TP
Externe (OS)	`IDENTIFIED EXTERNALLY`	Compte `OPS$` lié à l'OS
Global	`IDENTIFIED GLOBALLY`	Annuaire LDAP / Kerberos
Aucune	`NO AUTHENTICATION`	Compte schéma seul (propriétaire d'application)

➤ Exemple — Mot de passe et casse

Depuis la 11g, les mots de passe sont **sensibles à la casse**. Entre guillemets, ils peuvent contenir des caractères spéciaux : `IDENTIFIED BY "Lina#2026"`.

✎ Attention !

Le **nom** d'utilisateur, lui, est converti en majuscules : `lina` devient `LINA` dans `DBA_USERS`. Pensez-y dans vos clauses `WHERE`.

1.2 Tablespaces et quotas

★ À retenir — Le quota

Un utilisateur ne peut créer de segment (table, index) que dans un tablespace où il dispose d'un **quota** (ou du privilège `UNLIMITED TABLESPACE`, à éviter). Le quota se modifie à tout moment :

```
ALTER USER lina QUOTA 50M ON users;   ALTER USER lina QUOTA 0 ON users;
```

✎ Piège : la création différée des segments

En 19c, `CREATE TABLE` ne réserve aucun espace tant que la table est vide (*deferred segment creation*). Une table peut donc être créée sans quota ; l'erreur `ORA-01950` n'apparaît qu'au **premier INSERT**.

1.3 États d'un compte

ACCOUNT_STATUS	Signification
OPEN	Compte utilisable
LOCKED	Verrouillé par le DBA (ACCOUNT LOCK)
LOCKED(TIMED)	Verrouillé automatiquement après trop d'échecs de connexion (profil)
EXPIRED	Mot de passe expiré : changement exigé à la connexion
EXPIRED(GRACE)	Période de grâce : connexion possible, avertissement ORA-28002
EXPIRED & LOCKED	Combinaison des deux états

1.4 Utilisateurs communs et locaux

	Utilisateur commun	Utilisateur local
Créé dans	CDB$ROOT	Une PDB
Nom	Préfixe C## obligatoire	Sans préfixe C##
Existe dans	Tous les conteneurs (CONTAINER=ALL)	Sa seule PDB
Exemples	SYS, SYSTEM, C##SUPERVISEUR	LINA, RH

● Comptes schéma seul et connexion proxy

Un compte NO AUTHENTICATION possède des objets mais personne ne peut s'y connecter avec un mot de passe : il ne peut pas être piraté par force brute. Pour déployer du code dans ce schéma, un développeur se connecte **à travers** lui grâce à l'authentification proxy :

`ALTER USER rh GRANT CONNECT THROUGH dev1;` puis `CONNECT dev1[rh]/mdp@orclpdb`

1.5 Exercices

Exercice 1.1 — *Premier compte*

1. **[ADMIN]** Créez l'utilisateur ALI (mot de passe Ali#2026, tablespace par défaut USERS, temporaire TEMP) sans aucune autre clause.
2. **[USER]** Tentez de vous connecter en ALI. Notez le message d'erreur et expliquez-le.
3. **[ADMIN]** Corrigez le problème avec le privilège minimal, puis reconnectez-vous.
4. Affichez USERNAME, DEFAULT_TABLESPACE, TEMPORARY_TABLESPACE, PROFILE, ACCOUNT_STATUS et CREATED pour ALI.

Exercice 1.2 — *Quotas et création différée*

1. **[ADMIN]** Accordez à ALI le privilège CREATE TABLE.
2. **[USER]** En ALI, créez la table notes(id NUMBER, texte VARCHAR2(4000)). La création

réussit-elle ? Insérez une ligne. Que se passe-t-il, et pourquoi la création avait-elle réussi ?

3. **[ADMIN]** Donnez à `ALI` un quota de `1M` sur `USERS`. Réessayez l'insertion.
4. **[USER]** Exécutez le bloc ci-dessous jusqu'à l'erreur. Quel code obtenez-vous ?

```
SQL*Plus
BEGIN
  FOR i IN 1..2000 LOOP
    INSERT INTO notes VALUES (i, RPAD('x', 4000, 'x'));
  END LOOP;
  COMMIT;
END;
/
```

5. **[ADMIN]** Affichez `BYTES` et `MAX_BYTES` de `DBA_TS_QUOTAS` pour `ALI`. Que signifierait `MAX_BYTES = -1` ?

Exercice 1.3 — *Cycle de vie d'un compte*

1. **[ADMIN]** Faites expirer le mot de passe d'`ALI`. Reconnectez-vous en `ALI` : que propose SQL*Plus ? Changez le mot de passe en `Ali#2027`.
2. **[ADMIN]** Verrouillez le compte. Tentez une connexion. Relevez le code d'erreur.
3. Affichez `ACCOUNT_STATUS`, `LOCK_DATE` et `EXPIRY_DATE` après chaque étape.
4. **[ADMIN]** Déverrouillez le compte. Pourquoi est-il préférable de *verrouiller* le compte d'un employé qui quitte l'entreprise plutôt que de le supprimer immédiatement ?

Exercice 1.4 — *Compte schéma seul et proxy*

1. Essayez de vous connecter en `RH` (n'importe quel mot de passe). Quel est le message ?
2. **[ADMIN]** Créez `DEV1` (mot de passe `Dev1#2026`, `CREATE SESSION`) et autorisez-le à se connecter à travers `RH`. Si la connexion proxy échoue, lisez attentivement le code d'erreur : de quel compte manque-t-il un privilège ?
3. Connectez-vous avec `dev1[rh]`, puis affichez `USER`, `SYS_CONTEXT('USERENV','PROXY_USER')` et la liste des tables du schéma courant.
4. Quel intérêt ce mécanisme présente-t-il pour la sécurité et pour la traçabilité ?

Exercice 1.5 — *Utilisateurs communs et locaux*

1. **[ADMIN]** Connectez-vous à la racine (`CONNECT / AS SYSDBA` sur le serveur). Tentez `CREATE USER bob IDENTIFIED BY "Bob#2026";`. Message ?
2. Créez l'utilisateur commun `C##SUPERVISEUR` dans tous les conteneurs et donnez-lui `CREATE SESSION` partout.
3. Revenez dans `ORCLPDB`. Tentez `CREATE USER c##test IDENTIFIED BY "Test#2026";`. Message ?

4. Affichez `USERNAME` et `COMMON` de `DBA_USERS` pour `ALI` et `C##SUPERVISEUR`.

Exercice 1.6 — *Supprimer un utilisateur*

1. **[ADMIN]** Tentez `DROP USER ali;`. Pourquoi la commande échoue-t-elle ?
2. Supprimez `ALI` avec la bonne option. Que deviennent sa table `NOTES` et son quota ?
3. Une session d'`ALI` est encore ouverte : quelle erreur obtiendriez-vous, et comment la terminer ?

Points de contrôle

Codes à retrouver dans votre compte rendu : `ORA-01045` (pas de `CREATE SESSION`) • `ORA-01950` (pas de quota) • `ORA-01536` (quota dépassé) • `ORA-28001` (mot de passe expiré) • `ORA-28000` (compte verrouillé) • `ORA-65096` / `ORA-65094` (nom commun / local invalide) • `ORA-01922` (`CASCADE` requis).

2 Privilèges

Dans cette partie, vous donnez aux utilisateurs le droit d'agir. Oracle distingue deux familles de privilèges : ceux qui portent sur la *base* (créer une table, se connecter) et ceux qui portent sur un *objet* précis (lire la table `RH.EMPLOYES`).

- ❶ **Accorder des privilèges système**
- ❷ **Accorder des privilèges objet, y compris par colonne**
- ❸ **Comprendre ADMIN OPTION et GRANT OPTION**
- ❹ **Mesurer le danger des privilèges ANY**
- ❺ **Interroger le dictionnaire**

● Le saviez-vous ?

Oracle 19c compte plus de **200 privilèges système**. Un seul, mal placé, peut suffire à ouvrir toute la base : `SELECT ANY TABLE` donne accès en lecture aux tables de *tous* les schémas applicatifs.

Atelier 2.1 — Système ou objet ?

	Privilège système	Privilège objet
Porte sur	une action dans la base	un objet nommé
Exemple	`GRANT CREATE TABLE TO dev1;`	`GRANT SELECT ON rh.employes TO lina;`
Retransmission	`WITH ADMIN OPTION`	`WITH GRANT OPTION`
Révocation en cascade	non	oui
Vues	`DBA_SYS_PRIVS`	`DBA_TAB_PRIVS`, `DBA_COL_PRIVS`

2.1 Privilèges système

Privilège	Autorise
`CREATE SESSION`	la connexion
`CREATE TABLE / VIEW / SEQUENCE`	la création de ces objets *dans son propre schéma*
`CREATE PROCEDURE / TRIGGER`	la création de code PL/SQL dans son schéma
`UNLIMITED TABLESPACE`	un espace illimité dans tous les tablespaces (à proscrire)
`CREATE USER / ALTER USER`	l'administration des comptes
`SELECT ANY TABLE`	la lecture de toute table de tout schéma (hors dictionnaire)
`SELECT ANY DICTIONARY`	la lecture des tables du dictionnaire (schéma `SYS`)

✎ Les privilèges ANY

Tout privilège contenant le mot `ANY` s'applique à **tous les schémas**. Il se réserve aux administrateurs et aux outils, jamais à un compte applicatif.

2.2 Privilèges objet

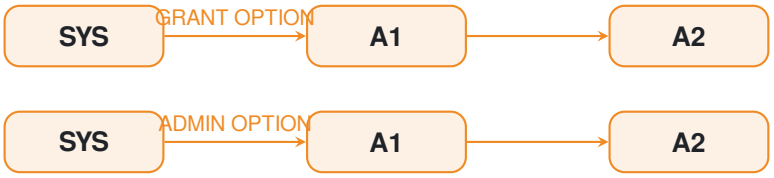
Privilège	Objets	Remarque
`SELECT`	table, vue, séquence	
`READ`	table, vue	lecture sans `SELECT ... FOR UPDATE` (12c et +)
`INSERT, UPDATE`	table, vue	peuvent être limités à des colonnes
`DELETE`	table, vue	
`EXECUTE`	procédure, fonction, package	
`REFERENCES`	table	créer une clé étrangère vers la table (colonnes possibles)

➤ Exemple — Privilèges sur colonnes

```
GRANT UPDATE (poste, dept_id) ON rh.employes TO omar;
```

Omar peut modifier le poste et le département, pas le salaire. Pour restreindre la **lecture** à certaines colonnes, on passe par une **vue** (`RH.V_ANNUAIRE`).

2.3 Transmettre un privilège



Objet : on révoque A1 ⇒ A2 **perd** le privilège (cascade)

Système : on révoque A1 ⇒ A2 **garde** le privilège

★ À retenir — Vues utiles

`SESSION_PRIVS` (mes privilèges système actifs) • `USER_TAB_PRIVS_RECD` / `_MADE` (reçus / accordés) • `DBA_SYS_PRIVS`, `DBA_TAB_PRIVS`, `DBA_COL_PRIVS` (vue globale, colonne `GRANTOR` pour savoir qui a accordé).

2.4 Exercices

Exercice 2.1 — *Le développeur minimal*

[ADMIN] Le développeur `DEV1` (créé au chapitre 1) doit pouvoir, *dans son seul schéma*, créer des tables, des vues, des séquences et des procédures, avec 20 Mo sur `USERS`.

1. Accordez exactement ce qui est nécessaire (aucun rôle, aucun privilège `ANY`).
2. **[USER]** Connecté en `DEV1`, affichez `SESSION_PRIVS`. Créez une table `projets`, une séquence et une vue sur cette table.
3. Pourquoi ne pas donner `UNLIMITED TABLESPACE` à la place du quota ?

Exercice 2.2 — *Lire sans modifier*

1. **[ADMIN]** Créez `LECTEUR1` (`Lect1#2026`, `CREATE SESSION`) et accordez-lui la lecture de `RH.EMPLOYES`.
2. **[USER]** En `LECTEUR1` : lisez `rh.employes`, puis tentez `UPDATE rh.employes SET salaire = 1;`, puis `SELECT * FROM rh.departements;`. Comparez les deux codes d'erreur : pourquoi sont-ils différents ?
3. **[ADMIN]** Remplacez le privilège `SELECT` par `READ`. En `LECTEUR1`, testez un `SELECT` puis un `SELECT ... FOR UPDATE`. Conclusion ?

Exercice 2.3 — *Droits par colonne, SQL92_SECURITY et vue*

1. **[ADMIN]** Créez `OMAR` (`Omar#2026`, `CREATE SESSION`). Il doit pouvoir modifier le poste et le département des employés, mais pas leur salaire. Accordez-lui `UPDATE` sur ces deux colonnes de `RH.EMPLOYES`.
2. **[USER]** En `OMAR`, testez successivement (puis `ROLLBACK`) :

```
UPDATE rh.employes SET salaire = 99999;
UPDATE rh.employes SET poste = 'Architecte' WHERE emp_id = 3;
UPDATE rh.employes SET poste = 'Architecte';
```

Le deuxième ordre échoue alors qu'il ne touche que la colonne autorisée. Affichez le paramètre `SQL92_SECURITY` et expliquez.

3. Accorder `SELECT` sur la table résoudrait le problème… mais Omar verrait les salaires. Proposez une solution qui passe par la vue `RH.V_ANNUAIRE`, appliquez-la et retirez les droits sur la table.
4. Vérifiez dans `DBA_COL_PRIVS` et `DBA_TAB_PRIVS` ce que possède désormais `OMAR`.

Exercice 2.4 — *GRANT OPTION : la cascade*

1. **[ADMIN]** Créez `A1` et `A2` (`CREATE SESSION`). Accordez à `A1` la lecture de `RH.DEPARTEMENTS` **avec** possibilité de la retransmettre.
2. En `A1`, accordez la même lecture à `A2`. Vérifiez dans `DBA_TAB_PRIVS` (colonnes `GRANTEE`, `GRANTOR`, `GRANTABLE`).
3. **[ADMIN]** Retirez le privilège à `A1`. `A2` peut-il encore lire la table ? Expliquez.

Exercice 2.5 — *ADMIN OPTION : pas de cascade*

1. **[ADMIN]** Accordez `CREATE VIEW` à `A1` **avec** `ADMIN OPTION`. En `A1`, accordez `CREATE VIEW` à `A2`.
2. **[ADMIN]** Révoquez `CREATE VIEW` de `A1`. `A2` possède-t-il encore ce privilège ? Vérifiez dans `DBA_SYS_PRIVS`.
3. Résumez dans un tableau la différence entre les deux options. Laquelle est la plus dangereuse pour un auditeur, et pourquoi ?

Exercice 2.6 — *Le danger des privilèges ANY*

1. **[ADMIN]** Accordez `SELECT ANY TABLE` à `A2`.
2. En `A2` : lisez `rh.employes`, puis `dev1.projets`, puis `sys.user$`. Quels accès réussissent ?
3. **[ADMIN]** Listez tous les utilisateurs (hors comptes Oracle) qui possèdent un privilège système contenant `ANY`, directement. Révoquez celui d'`A2`.

Points de contrôle

À savoir expliquer : `ORA-00942` contre `ORA-01031` • `READ` contre `SELECT` • colonnes pour `INSERT`/`UPDATE`/`REFERENCES`, vue pour la lecture • cascade pour `GRANT OPTION`, pas pour `ADMIN OPTION` • privilèges `ANY` réservés aux administrateurs.

3 Rôles et modèle RBAC

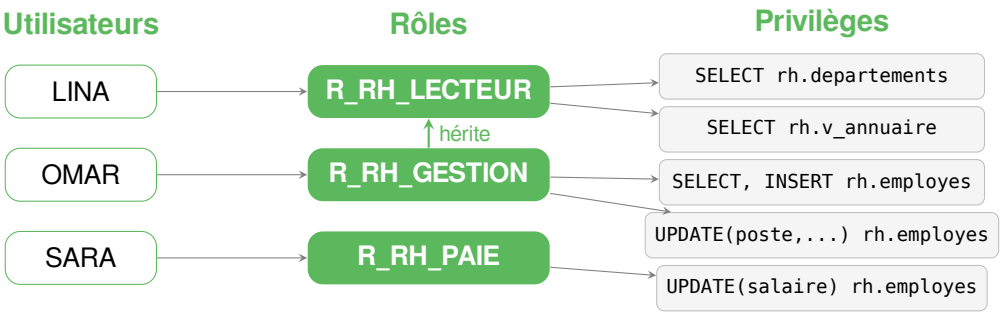
Dans cette partie, vous passez d'une gestion « personne par personne » à une gestion **par fonction**. Les privilèges sont regroupés dans des rôles qui reflètent les métiers ; les utilisateurs reçoivent des rôles.

- ➤ ❶ **Concevoir une matrice rôles / privilèges**
- ➤ ❷ **Construire une hiérarchie de rôles**
- ➤ ❸ **Gérer rôles par défaut et rôles protégés**
- ➤ ❹ **Connaître les limites des rôles (PL/SQL, vues)**
- ➤ ❺ **Mesurer les privilèges réellement utilisés**

● Le saviez-vous ?

Sans rôles, 50 utilisateurs × 20 privilèges = 1 000 `GRANT` à maintenir. Avec 4 rôles : 80 `GRANT` vers les rôles et 50 `GRANT` de rôles. Et l'arrivée d'un nouvel employé se résume à **une seule commande**.

Atelier 3.1 — Le modèle RBAC en une image



1. On accorde des privilèges aux rôles, jamais (ou presque) aux personnes.
2. Un rôle peut recevoir un autre rôle : il en **hérite** tous les privilèges.
3. On accorde aux personnes les rôles correspondant à leur fonction.

3.1 Créer et attribuer des rôles

✻ Définition — Rôle

Un **rôle** est un ensemble nommé de privilèges (système, objet) et d'autres rôles. Il n'appartient à aucun schéma. Il peut être accordé à un utilisateur ou à un autre rôle, et activé ou désactivé dans une session.

[ADMIN] syntaxe

```
CREATE ROLE r_rh_lecteur;                          -- rôle simple
CREATE ROLE r_rh_paie IDENTIFIED BY "Paie#2026";   -- rôle protégé par mot de passe
GRANT SELECT ON rh.departements TO r_rh_lecteur;   -- privilège -> rôle
GRANT r_rh_lecteur TO r_rh_gestion;                -- rôle -> rôle (hiérarchie)
GRANT r_rh_gestion TO omar;                        -- rôle -> utilisateur
ALTER USER sara DEFAULT ROLE ALL EXCEPT r_rh_paie; -- rôles actifs à la connexion
SET ROLE r_rh_paie IDENTIFIED BY "Paie#2026";      -- activation dans la session
```

3.2 Quand une modification prend-elle effet ?

★ À retenir — Effet immédiat ou différé

- Un privilège accordé **à un rôle** déjà actif est utilisable **immédiatement** par les sessions ouvertes.
- Un rôle accordé **à un utilisateur** n'est actif qu'à sa **prochaine connexion** (ou après `SET ROLE`).
- Un rôle protégé par mot de passe accordé à un utilisateur devient un rôle **par défaut** : il est alors activé *sans* mot de passe à la connexion !

3.3 Rôles prédéfinis

Rôle	Contenu (19c) et conseil
`CONNECT`	`CREATE SESSION` et `SET CONTAINER` uniquement
`RESOURCE`	`CREATE TABLE, SEQUENCE, PROCEDURE, TRIGGER, TYPE…` — **plus** de `UNLIMITED TABLESPACE` depuis 12c
`DBA`	quasiment tous les privilèges système : à réserver aux administrateurs
`SELECT_CATALOG_ROLE`	lecture des vues du dictionnaire (`DBA_*`, `V$*`)
`AUDIT_ADMIN` / `AUDIT_VIEWER`	gérer / consulter l'audit unifié (chapitre 5)

✎ Les rôles ne traversent pas le code PL/SQL nommé

Dans une procédure, une fonction, un package ou un trigger en *droits du définisseur* (le cas par défaut), et pour créer une **vue**, Oracle ignore les rôles du propriétaire : seuls ses privilèges **accordés directement** comptent. C'est une source classique d'`ORA-00942` « inexplicables ».

★ À retenir — Vues des rôles

`DBA_ROLES` (rôles existants) • `DBA_ROLE_PRIVS` (rôles accordés aux utilisateurs et rôles, `DEFAULT_ROLE`, `ADMIN_OPTION`) • `ROLE_SYS_PRIVS`, `ROLE_TAB_PRIVS`, `ROLE_ROLE_PRIVS` (contenu des rôles) • `SESSION_ROLES` (rôles actifs dans ma session).

3.4 Exercices

Exercice 3.1 — *Construire la matrice RBAC du service RH*

Le service RH exprime les besoins suivants :

Fonction	Besoin	Titulaire
Lecteur	consulter les départements et l'annuaire (sans salaires)	`LINA`
Gestionnaire	tout ce que fait le lecteur + consulter et créer des employés + modifier poste et département	`OMAR`
Paie	consulter les employés et modifier *uniquement* les salaires	`SARA`

1. Remplissez sur papier la matrice rôles × privilèges (une ligne par rôle, une colonne par objet).
2. **[ADMIN]** Créez `LINA` et `SARA` (mots de passe `Lina#2026`, `Sara#2026`) si elles n'existent pas. Retirez à `OMAR` les privilèges objet accordés directement au chapitre 2.
3. Créez les rôles `R_RH_LECTEUR`, `R_RH_GESTION` (qui hérite du lecteur) et `R_RH_PAIE`, alimentez-les, puis attribuez-les. Le privilège `CREATE SESSION` sera porté par un rôle `R_CONNEXION` accordé aux trois rôles métier.
4. **[USER]** Connectez-vous successivement avec chaque compte et vérifiez `SESSION_ROLES` ainsi qu'un accès autorisé et un accès refusé.

Exercice 3.2 — *Lire la hiérarchie dans le dictionnaire*

1. Affichez les rôles accordés à `OMAR` (`DBA_ROLE_PRIVS`). Pourquoi `R_RH_LECTEUR` n'y apparaît-il pas ?
2. Écrivez une requête hiérarchique (`CONNECT BY`) qui affiche l'arbre complet des rôles d'`OMAR`, avec indentation.
3. Listez tous les privilèges objet dont `OMAR` dispose, directement ou via ses rôles (`ROLE_TAB_PRIVS` pour les rôles de l'arbre).

Exercice 3.3 — *Effet immédiat ou à la reconnexion ?*

1. Gardez une session `LINA` ouverte. **[ADMIN]** Accordez `SELECT ON rh.employes` au rôle `R_RH_LECTEUR`. Dans la session de Lina, relancez `SELECT COUNT(*) FROM rh.employes;`. Résultat ?
2. Annulez ce `GRANT`. Créez le rôle `R_TEST` portant `SELECT ON rh.employes` et accordez-le à `LINA`. Dans la même session : la requête réussit-elle ? Puis après `SET ROLE ALL;` ? Puis après reconnexion ?
3. Concluez : pourquoi faut-il, lors d'un départ, *aussi* tuer les sessions ouvertes après un `REVOKE` de rôle ?

Exercice 3.4 — *Rôle protégé et rôles par défaut*

La paie est sensible : Sara ne doit disposer de `R_RH_PAIE` qu'après avoir saisi un mot de passe spécifique.

1. **[ADMIN]** Protégez le rôle par le mot de passe `Paie#2026`. Sara se reconnecte : a-t-elle besoin du mot de passe ? Vérifiez `SESSION_ROLES` et expliquez.
2. Faites en sorte que seul `R_CONNEXION` soit actif à la connexion de Sara. Vérifiez la colonne `DEFAULT_ROLE` de `DBA_ROLE_PRIVS`.
3. **[USER]** Sara active le rôle avec un mauvais mot de passe, puis avec le bon. Notez l'erreur.

Exercice 3.5 — *Le piège des rôles dans le code*

1. **[ADMIN]** Accordez `CREATE PROCEDURE` et `CREATE VIEW` au rôle `R_RH_GESTION`, et un quota de 5 Mo à `OMAR`.
2. **[USER]** En `OMAR`, compilez la fonction ci-dessous. Affichez `SHOW ERRORS`. Puis tentez `CREATE VIEW v_effectif AS SELECT dept_id, COUNT(*) n FROM rh.employes GROUP BY dept_id;`

```sql
CREATE OR REPLACE FUNCTION nb_employes RETURN NUMBER AS
  n NUMBER;
BEGIN
  SELECT COUNT(*) INTO n FROM rh.employes;
  RETURN n;
END;
/
```

SQL*Plus

3. Omar lit pourtant `rh.employes` en SQL direct. Expliquez, puis corrigez de la manière la plus restreinte possible.

Exercice 3.6 — *Que contiennent CONNECT et RESOURCE ?*

1. Affichez le contenu des rôles `CONNECT` et `RESOURCE` (`ROLE_SYS_PRIVS`).
2. Créez `TEST_RES` avec `GRANT CONNECT, RESOURCE TO test_res;` puis, en `TEST_RES`, créez une table et insérez une ligne. Que manque-t-il ?
3. Pourquoi ces rôles ne doivent-ils pas servir de base à une politique RBAC d'entreprise ?

Exercice 3.7 — *Pour aller plus loin : l'analyse des privilèges*

Oracle 19c permet de capturer les privilèges **réellement utilisés** par un compte afin de retirer les autres.

1. **[ADMIN]** Créez et démarrez une capture limitée aux sessions d'`OMAR` avec le package `DBMS_PRIVILEGE_CAPTURE`.
2. **[USER]** Omar lit l'annuaire et modifie un poste (puis `ROLLBACK`).
3. **[ADMIN]** Arrêtez la capture, générez les résultats et comparez `DBA_USED_OBJPRIVS` et `DBA_UNUSED_OBJPRIVS`. Quels privilèges pourriez-vous retirer ?

Points de contrôle

À savoir expliquer : rôle → rôle → utilisateur • `SESSION_ROLES` contre `DBA_ROLE_PRIVS` • `DEFAULT ROLE ... EXCEPT` et `SET ROLE` • `ORA-01979` (mot de passe de rôle) • rôles ignorés dans le PL/SQL nommé et les vues.

4 Profils

Dans cette partie, vous fixez des règles de vie aux comptes : solidité et durée des mots de passe, verrouillage après échecs, nombre de sessions, temps d'inactivité. C'est le *règlement intérieur* de la base.

- ❶ **Créer un profil et l'attribuer**
- ❷ **Limiter les ressources d'une session**
- ❸ **Imposer une politique de mots de passe**
- ❹ **Contrôler la complexité des mots de passe**
- ❺ **Observer les effets dans le dictionnaire**

● Le saviez-vous ?

Tout utilisateur a **exactement un** profil. Sans précision, c'est le profil `DEFAULT`. Un paramètre fixé à `DEFAULT` dans un profil reprend la valeur du profil `DEFAULT`.

Atelier 4.1 — Un profil complet

[ADMIN]

```
CREATE PROFILE p_rh_standard LIMIT
  SESSIONS_PER_USER        2          -- sessions simultanées
  IDLE_TIME                15         -- minutes d'inactivité
  CONNECT_TIME             480        -- minutes de connexion
  FAILED_LOGIN_ATTEMPTS    3          -- échecs avant verrouillage
  PASSWORD_LOCK_TIME       1/24       -- jours de verrouillage (1 heure)
  PASSWORD_LIFE_TIME       90         -- jours de validité
  PASSWORD_GRACE_TIME      7          -- jours de grâce
  PASSWORD_REUSE_TIME      365        -- jours avant réutilisation...
  PASSWORD_REUSE_MAX       5          -- ...et après 5 changements
  PASSWORD_VERIFY_FUNCTION ora12c_verify_function;
ALTER USER lina PROFILE p_rh_standard;
```

4.1 Limites de ressources

Paramètre	Unité	Effet en cas de dépassement
`SESSIONS_PER_USER`	sessions	connexion refusée, `ORA-02391`
`IDLE_TIME`	minutes	session coupée, `ORA-02396` à l'appel suivant
`CONNECT_TIME`	minutes	session coupée, `ORA-02399`
`CPU_PER_CALL`	centièmes de s	appel interrompu, `ORA-02393`
`LOGICAL_READS_PER_SESSION`	blocs	session interrompue, `ORA-02394`

✎ Le paramètre RESOURCE_LIMIT

Les limites de ressources ne sont appliquées que si `RESOURCE_LIMIT = TRUE` (valeur par défaut depuis 12c). Les paramètres de mot de passe, eux, s'appliquent toujours.

4.2 Politique de mots de passe

Paramètre	Rôle
`FAILED_LOGIN_ATTEMPTS`	nombre d'échecs consécutifs avant verrouillage
`PASSWORD_LOCK_TIME`	durée du verrouillage automatique (jours, fractions possibles)
`PASSWORD_LIFE_TIME`	durée de validité du mot de passe
`PASSWORD_GRACE_TIME`	délai de grâce après expiration (`ORA-28002`)
`PASSWORD_REUSE_TIME / _MAX`	interdiction de réutiliser un ancien mot de passe (les deux doivent être fixés)
`PASSWORD_VERIFY_FUNCTION`	fonction PL/SQL qui valide la complexité
`INACTIVE_ACCOUNT_TIME`	verrouillage d'un compte inutilisé depuis n jours (18c et +, minimum 15)

★ À retenir — Les fonctions de vérification fournies

Le script `$ORACLE_HOME/rdbms/admin/utlpwdmg.sql` crée les fonctions `ORA12C_VERIFY_FUNCTION` (8 caractères minimum, au moins une lettre et un chiffre, différent du nom d'utilisateur, 3 caractères différents de l'ancien mot de passe) et `ORA12C_STRONG_VERIFY_FUNCTION` (plus stricte). **Attention :** ce script modifie aussi le profil `DEFAULT`.

➤ Exemple — Changer son propre mot de passe sous une fonction de vérification

```
ALTER USER lina IDENTIFIED BY "Nouveau#2027" REPLACE "Lina#2026";
```

La clause `REPLACE` fournit l'ancien mot de passe, que la fonction compare au nouveau.

4.3 Exercices

Exercice 4.1 — *Profil de ressources*

1. **[ADMIN]** Vérifiez la valeur de `RESOURCE_LIMIT`. Créez le profil `P_RH_TEST` avec `SESSIONS_PER_USER 2` et `IDLE_TIME 1`, puis attribuez-le à `LINA`.
2. Ouvrez trois sessions `LINA` en parallèle. Que se passe-t-il à la troisième ?
3. Laissez une session `LINA` inactive deux à trois minutes, puis exécutez une requête. Message ? Que voit le DBA dans `V$SESSION` (colonne `STATUS`) pendant l'attente ?
4. Affichez les paramètres de `P_RH_TEST` dans `DBA_PROFILES`. Que signifie `DEFAULT` dans la colonne `LIMIT` ?

Exercice 4.2 — *Verrouillage après échecs*

1. **[ADMIN]** Modifiez `P_RH_TEST` : 3 échecs autorisés, verrouillage de 2 minutes (`2/1440`).
2. Tentez trois connexions `LINA` avec un mauvais mot de passe, puis une quatrième avec le bon. Message ?
3. Affichez `ACCOUNT_STATUS` et `LOCK_DATE`. Attendez deux minutes et reconnectez-vous. Que s'est-il passé ?
4. Quel risque présente une valeur `FAILED_LOGIN_ATTEMPTS UNLIMITED` ? Et une valeur trop basse pour un compte applicatif ?

Exercice 4.3 — *Expiration, grâce et réutilisation*

1. **[ADMIN]** Fixez dans `P_RH_TEST` : `PASSWORD_REUSE_TIME 30`, `PASSWORD_REUSE_MAX 3`.
2. **[USER]** En `LINA`, changez votre mot de passe, puis tentez de reprendre l'ancien. Message ?
3. **[ADMIN]** Fixez `PASSWORD_LIFE_TIME 1/1440` et `PASSWORD_GRACE_TIME 1/1440`, puis changez le mot de passe de Lina. Reconnectez-vous en Lina après une minute, puis après deux minutes. Décrivez les états `EXPIRED(GRACE)` puis `EXPIRED`.
4. Rétablissez `PASSWORD_LIFE_TIME 90` et `PASSWORD_GRACE_TIME 7`.

Exercice 4.4 — *Complexité des mots de passe*

1. **[ADMIN]** Vérifiez si la fonction `ORA12C_VERIFY_FUNCTION` existe (`DBA_OBJECTS`, propriétaire `SYS`). Si ce n'est pas le cas, notez les valeurs actuelles du profil `DEFAULT`, exécutez `@?/rdbms/admin/utlpwdmg.sql` dans la PDB, puis restaurez le profil `DEFAULT`.
2. Associez la fonction à `P_RH_TEST`. Tentez de donner à Lina les mots de passe `lina`, `abcdefgh`, `Lina2026x`. Lesquels sont refusés et pourquoi ?
3. **[USER]** En `LINA`, changez votre mot de passe sans la clause `REPLACE`, puis avec. Différence ?

Exercice 4.5 — *Profil de production et synthèse*

1. Créez le profil `P_RH_STANDARD` de l'atelier 4.1 et attribuez-le à `LINA`, `OMAR` et `SARA`. Supprimez `P_RH_TEST`. Que devient l'utilisateur d'un profil supprimé avec `CASCADE` ?
2. Affichez, pour les utilisateurs non maintenus par Oracle, leur profil, l'état du compte et la date d'expiration.
3. Proposez un profil `P_SERVICE` adapté à un compte applicatif (batch) : quels paramètres assouplir, lesquels durcir ? Justifiez.

Points de contrôle

Codes à retrouver : `ORA-02391` (sessions) • `ORA-02396` (inactivité) • `ORA-28000` (verrouillé) • `ORA-28002` (grâce) • `ORA-28001` (expiré) • `ORA-28007` (réutilisation) • `ORA-28003` (complexité) • `ORA-28221` (`REPLACE` manquant).

5 Audit unifié et FGA

Dans cette partie, vous rendez la sécurité *vérifiable*. Les privilèges disent ce qui est permis ; l'audit dit ce qui a été **fait**, par qui, quand et d'où. En 19c, l'audit se configure par des **politiques** et se lit dans une vue unique.

- ❶ **Vérifier le mode d'audit de la base**
- ❷ **Exploiter les politiques prédéfinies**
- ❸ **Créer des politiques sur objets, rôles et actions**
- ❹ **Auditer finement une valeur (FGA)**
- ❺ **Séparer les tâches et purger la trace**

● Le saviez-vous ?

Par défaut, une base 19c est en **mode mixte** : l'audit unifié fonctionne, et l'ancien audit (paramètre `AUDIT_TRAIL`) aussi. Le mode **pur**, recommandé, exige de relinker l'exécutable Oracle : il n'est pas nécessaire pour ce TP.

Atelier 5.1 — Le cycle de vie d'une politique

❶ **Créer** la politique : ce qu'on veut tracer.

```
[ADMIN]
CREATE AUDIT POLICY pol_rh_salaires
  ACTIONS SELECT, UPDATE, DELETE ON rh.employes;
```

❷ **Activer** la politique : pour qui, dans quel cas.

```
[ADMIN]
AUDIT POLICY pol_rh_salaires;                          -- pour tous
AUDIT POLICY pol_rh_salaires EXCEPT omar;              -- ou : sauf certains
AUDIT POLICY pol_rh_salaires WHENEVER NOT SUCCESSFUL;  -- ou : échecs seulement
```

❸ **Lire** la trace dans `UNIFIED_AUDIT_TRAIL`, puis **désactiver** (`NOAUDIT POLICY`) et **supprimer** (`DROP AUDIT POLICY`).

5.1 Ce qu'une politique peut tracer

Clause	Exemple	Trace
PRIVILEGES	`PRIVILEGES SELECT ANY TABLE, CREATE USER`	l'usage d'un privilège système
ACTIONS	`ACTIONS DELETE ON rh.employes` `ACTIONS CREATE TABLE, DROP TABLE`	une action sur un objet ou une action globale
ROLES	`ROLES dba`	tout usage des privilèges système du rôle
WHEN	`WHEN 'SYS_CONTEXT(...) ...' EVALUATE PER SESSION`	condition d'application

Politique prédéfinie	Contenu
`ORA_LOGON_FAILURES`	échecs de connexion (activée par défaut)
`ORA_SECURECONFIG`	actions sensibles de configuration (activée par défaut)
`ORA_ACCOUNT_MGMT`	création, modification, suppression d'utilisateurs et de rôles, `GRANT`, `REVOKE`
`ORA_DATABASE_PARAMETER`	modifications de paramètres d'initialisation

5.2 Audit fin (FGA)

✻ Définition — Fine-Grained Auditing

Le FGA, piloté par le package `DBMS_FGA`, trace un accès seulement si une **condition sur les données** est remplie (« lecture d'un salaire supérieur à 30 000 ») et éventuellement si certaines **colonnes** sont concernées. L'audit unifié classique, lui, trace l'action quelle que soit la ligne lue.

5.3 Lire, protéger et purger la trace

★ À retenir — Colonnes utiles de UNIFIED_AUDIT_TRAIL

`EVENT_TIMESTAMP` • `DBUSERNAME` • `ACTION_NAME` • `OBJECT_SCHEMA`, `OBJECT_NAME` • `SQL_TEXT` • `RETURN_CODE` (0 = succès) • `UNIFIED_AUDIT_POLICIES` • `FGA_POLICY_NAME` • `OS_USERNAME`, `USERHOST`, `CLIENT_PROGRAM_NAME`.

✎ Séparation des tâches

Les rôles `AUDIT_ADMIN` (gérer les politiques, purger) et `AUDIT_VIEWER` (lire la trace) permettent de confier l'audit à une personne qui **n'est pas** DBA. Un DBA ne devrait pas pouvoir effacer la trace de ses propres actions sans que cela soit lui-même tracé.

5.4 Exercices

Exercice 5.1 — *État de l'audit*

1. **[ADMIN]** L'option `Unified Auditing` est-elle à `TRUE` dans `V$OPTION` ? Qu'en déduisez-vous sur le mode (mixte ou pur) ?
2. Listez les politiques actives (`AUDIT_UNIFIED_ENABLED_POLICIES`) avec leurs colonnes `ENTITY_NAME`, `SUCCESS`, `FAILURE`.
3. Combien de politiques existent au total (`AUDIT_UNIFIED_POLICIES`, valeurs distinctes de `POLICY_NAME`) ?

Exercice 5.2 — *Échecs de connexion*

1. Tentez deux connexions `OMAR` avec un mauvais mot de passe et une connexion avec un utilisateur inexistant `PIRATE`.
2. **[ADMIN]** Retrouvez ces tentatives dans `UNIFIED_AUDIT_TRAIL` : heure, utilisateur, poste client, programme, code retour.
3. Quel code retour correspond à un mauvais mot de passe ? Quelle politique a produit ces enregistrements ?

Exercice 5.3 — *Tracer l'administration des comptes*

1. **[ADMIN]** Activez la politique prédéfinie `ORA_ACCOUNT_MGMT`.
2. Créez l'utilisateur `STAGIAIRE`, accordez-lui `R_RH_LECTEUR`, puis retirez-le et supprimez l'utilisateur.
3. Reconstituez cette histoire à partir de la trace : `ACTION_NAME`, `SQL_TEXT`, `DBUSERNAME`. Les mots de passe apparaissent-ils dans `SQL_TEXT` ?

Exercice 5.4 — *Politique sur les données sensibles*

1. **[ADMIN]** Créez et activez la politique `POL_RH_SALAIRES` de l'atelier 5.1 (pour tous).
2. En `SARA` (rôle de paie activé), lisez les employés et augmentez le salaire de l'employé 7 de 2 %. En `LINA`, lisez l'annuaire. En `OMAR`, lisez `rh.employes`.
3. Affichez les enregistrements produits par la politique. Qui a fait quoi ? L'accès de Lina via la vue apparaît-il ? Sous quel nom d'objet ?
4. Modifiez l'activation pour ne tracer que les **échecs** (désactivez, puis réactivez avec `WHENEVER NOT SUCCESSFUL`). Vérifiez dans `AUDIT_UNIFIED_ENABLED_POLICIES`.

Exercice 5.5 — *Politique conditionnelle et rôle*

1. **[ADMIN]** Créez `POL_RH_ADHOC` qui trace `INSERT`, `UPDATE`, `DELETE` sur `rh.employes` **uniquement** lorsque le programme client est SQL*Plus

(`SYS_CONTEXT('USERENV','CLIENT_PROGRAM_NAME')`), évalué une fois par session.

2. Créez `POL_DBA` qui trace l'usage des privilèges du rôle `DBA` et activez-la. Accordez temporairement `DBA` à `A1`, faites-lui créer un utilisateur, puis révoquez le rôle. Retrouvez l'action dans la trace.
3. Pourquoi auditer un *rôle* plutôt qu'une liste de privilèges ?

Exercice 5.6 — *Audit fin des gros salaires (FGA)*

1. **[ADMIN]** Créez une politique FGA `FGA_GROS_SALAIRES` sur `rh.employes` : elle trace les `SELECT` et `UPDATE` qui portent sur la colonne `SALAIRE` **et** concernent une ligne où `salaire > 30000`.
2. En `SARA` (après `SET ROLE r_rh_paie IDENTIFIED BY ...`) : (a) lisez les noms seuls ; (b) lisez les salaires du département 40 ; (c) lisez les salaires du département 10. Quelles requêtes sont tracées ?
3. Retrouvez les enregistrements : interrogez `UNIFIED_AUDIT_TRAIL` (colonne `FGA_POLICY_NAME`) et `DBA_FGA_AUDIT_TRAIL`. Dans quelle vue apparaissent-ils dans votre base, et pourquoi ?
4. Listez la politique dans `DBA_AUDIT_POLICIES`, puis désactivez-la avec `DBMS_FGA.DISABLE_POLICY`.

Exercice 5.7 — *Séparation des tâches et purge*

1. **[ADMIN]** Créez l'utilisateur `AUDITEUR` (`CREATE SESSION` + `AUDIT_VIEWER`). En `AUDITEUR`, lisez la trace, puis tentez `NOAUDIT POLICY ora_logon_failures;`. Résultat ?
2. Écrivez un rapport : nombre d'événements par utilisateur et par action sur les dernières 24 heures.
3. **[ADMIN]** Désactivez et supprimez `POL_RH_ADHOC` et `POL_DBA`. Peut-on supprimer une politique encore active ?
4. Purgez la trace unifiée avec `DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL`. Que trouvez-vous ensuite dans la trace ? Pourquoi est-ce rassurant ?

Points de contrôle

À savoir expliquer : `CREATE AUDIT POLICY` puis `AUDIT POLICY` • `BY` / `EXCEPT` / `WHENEVER [NOT] SUCCESSFUL` • politiques prédéfinies • FGA = condition sur les valeurs • `AUDIT_ADMIN` contre `AUDIT_VIEWER` • la purge est elle-même auditée.

6 Étude de cas : Banque Atlas

Dans cette étude de cas, vous êtes le DBA de la **Banque Atlas**, agence de Tanger. La direction vous confie la mise en sécurité de la base des comptes clients. Vous devez réunir tout ce que vous avez pratiqué : comptes, privilèges, rôles, profils et audit.

- ❶ **Analyser un besoin métier**
- ❷ **Concevoir la matrice RBAC**
- ❸ **Implémenter comptes, rôles et profils**
- ❹ **Mettre en place l'audit**
- ❺ **Prouver que la politique fonctionne**

Durée et livrables

Durée conseillée : **4 heures**. Livrables : un script SQL commenté par tâche (`T1.sql` à `T7.sql`), le tableau de tests rempli, et un rapport de deux pages répondant aux questions de synthèse.

6.1 Le contexte

La base contient trois tables dans le schéma `BANQUE` : les clients, leurs comptes et les opérations de guichet. Les mouvements d'argent ne doivent **jamais** être saisis directement dans les tables : ils passent par la procédure `BANQUE.P_OPERATION`, qui contrôle le solde, met à jour le compte et journalise l'opération.

[ADMIN] 10-init-banque.sql

```
CREATE USER banque NO AUTHENTICATION DEFAULT TABLESPACE users QUOTA UNLIMITED ON users;

CREATE TABLE banque.clients (
  client_id      NUMBER PRIMARY KEY,
  nom            VARCHAR2(60) NOT NULL,
  cin            VARCHAR2(12) UNIQUE,
  telephone      VARCHAR2(20),
  revenu_mensuel NUMBER(10,2),
  conseiller     VARCHAR2(30));
```

[ADMIN] 10-init-banque.sql

```
CREATE TABLE banque.comptes (
  compte_id      NUMBER PRIMARY KEY,
  client_id      NUMBER NOT NULL REFERENCES banque.clients,
  type_compte    VARCHAR2(10) CHECK (type_compte IN ('COURANT','EPARGNE')),
  solde          NUMBER(12,2) DEFAULT 0 NOT NULL,
  date_ouverture DATE DEFAULT SYSDATE);

CREATE TABLE banque.operations (
  op_id     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  compte_id NUMBER NOT NULL REFERENCES banque.comptes,
  type_op   VARCHAR2(10) CHECK (type_op IN ('DEPOT','RETRAIT')),
  montant   NUMBER(12,2) NOT NULL,
  date_op   DATE DEFAULT SYSDATE,
  agent     VARCHAR2(30));

INSERT INTO banque.clients VALUES (1, 'Amrani Leila',   'K123456', '0661000001', 18000,
    'CONS1');
INSERT INTO banque.clients VALUES (2, 'Bouzid Mehdi',   'K234567', '0661000002',  9500,
    'CONS1');
INSERT INTO banque.clients VALUES (3, 'Cherkaoui Sara', 'L345678', '0661000003', 42000,
    'CONS1');
INSERT INTO banque.clients VALUES (4, 'Daoudi Anas',    'K456789', '0661000004',  6000,
    'CONS1');
INSERT INTO banque.comptes VALUES (1001, 1, 'COURANT',  25000, DATE '2020-01-10');
INSERT INTO banque.comptes VALUES (1002, 1, 'EPARGNE', 150000, DATE '2020-01-10');
INSERT INTO banque.comptes VALUES (1003, 2, 'COURANT',   4200, DATE '2021-06-01');
INSERT INTO banque.comptes VALUES (1004, 3, 'COURANT',  98000, DATE '2019-03-15');
INSERT INTO banque.comptes VALUES (1005, 4, 'COURANT',   1500, DATE '2023-09-30');
COMMIT;

-- Vue du guichet : ni revenu, ni conseiller
CREATE VIEW banque.v_clients_guichet AS
  SELECT client_id, nom, cin, telephone FROM banque.clients;

-- Seule porte d'entrée pour les mouvements d'argent (droits du définisseur)
CREATE OR REPLACE PROCEDURE banque.p_operation (
  p_compte NUMBER, p_type VARCHAR2, p_montant NUMBER) AS
  v_solde banque.comptes.solde%TYPE;
BEGIN
  IF p_montant <= 0 OR p_type NOT IN ('DEPOT','RETRAIT') THEN
    RAISE_APPLICATION_ERROR(-20001, 'Opération invalide');
  END IF;
  SELECT solde INTO v_solde FROM banque.comptes WHERE compte_id = p_compte FOR UPDATE;
  IF p_type = 'RETRAIT' AND v_solde < p_montant THEN
    RAISE_APPLICATION_ERROR(-20002, 'Solde insuffisant');
  END IF;
  UPDATE banque.comptes
     SET solde = solde + CASE p_type WHEN 'DEPOT' THEN p_montant ELSE -p_montant END
   WHERE compte_id = p_compte;
  INSERT INTO banque.operations (compte_id, type_op, montant, agent)
  VALUES (p_compte, p_type, p_montant, SYS_CONTEXT('USERENV','SESSION_USER'));
```

```
  COMMIT;
END;
/
```
[ADMIN] 10-init-banque.sql

6.2 Les acteurs et leurs besoins

Fonction	Comptes	Besoins exprimés par la direction
Guichetier	GUI1, GUI2	identifier un client (nom, CIN, téléphone) ; consulter les comptes ; effectuer dépôts et retraits **uniquement** via P_OPERATION ; un seul poste à la fois
Conseiller	CONS1	tout ce que fait le guichetier ; consulter la fiche client complète et l'historique des opérations ; créer des clients ; modifier téléphone, revenu et conseiller (jamais le CIN) ; ouvrir des comptes
Directeur d'agence	DIR1	tout ce que fait le conseiller ; **habiliter** de nouveaux guichetiers lui-même, sans passer par le DBA
Auditeur interne	AUD1	lire toutes les tables et la trace d'audit ; ne rien modifier
Batch nocturne	APP_BATCH	verser les intérêts (dépôts via P_OPERATION) ; lire les comptes ; ne doit jamais être bloqué par une expiration de mot de passe
Développeur	DEV_BQ	déployer index et vues dans le schéma BANQUE sans en connaître de mot de passe

★ À retenir — Exigences de sécurité de la direction

- **E1.** Aucun privilège ne doit être accordé directement à un utilisateur métier, sauf exception justifiée.
- **E2.** Aucun compte métier ne possède de privilège ANY ni de rôle DBA, CONNECT ou RESOURCE.
- **E3.** Les soldes ne sont modifiables que par la procédure P_OPERATION.
- **E4.** Les agents (guichet, conseil) : 1 session (guichet) ou 2 (autres), 10 min d'inactivité, 3 échecs puis verrouillage 30 min, mot de passe valable 60 jours et non réutilisable avant un an.
- **E5.** Le directeur et l'auditeur ont un mot de passe **renforcé**.
- **E6.** Toute modification d'une fiche client est tracée ; toute tentative de modification directe d'un solde est tracée.
- **E7.** Tout mouvement d'un montant ≥ 50 000 MAD est tracé avec l'identité de l'agent.
- **E8.** La gestion des comptes et des habilitations (CREATE USER, GRANT...) est tracée.

6.3 Travail demandé

Exercice 6.1 — *T1 — Matrice RBAC*

Construisez la matrice *rôles* × *objets* (lignes : `R_BQ_CONNEXION`, `R_BQ_GUICHET`, `R_BQ_CONSEILLER`, `R_BQ_DIRECTEUR`, `R_BQ_AUDIT`, `R_BQ_BATCH` ; colonnes : les trois tables, la vue, la procédure, les privilèges système). Indiquez les héritages entre rôles et le seul privilège qui exige une option de retransmission. Justifiez chaque case au regard des exigences E1 à E3.

Exercice 6.2 — *T2 — Profils*

Créez `P_BQ_AGENT` (guichetiers et conseiller, E4), `P_BQ_SENSIBLE` (directeur et auditeur, E4 + E5 avec `ORA12C_STRONG_VERIFY_FUNCTION`) et `P_BQ_SERVICE` (batch). Les guichetiers ne doivent disposer que d'une session : comment gérez-vous la différence avec le conseiller (2 sessions) sans multiplier les profils ? Discutez.

Exercice 6.3 — *T3 — Comptes*

Créez `GUI1`, `GUI2`, `CONS1`, `DIR1`, `AUD1`, `APP_BATCH` et `DEV_BQ` avec le bon profil, aucun quota inutile et un mot de passe conforme à leur fonction de vérification. Configurez l'accès proxy de `DEV_BQ` au schéma `BANQUE`. Pourquoi `DEV_BQ` n'a-t-il pas besoin de quota ?

Exercice 6.4 — *T4 — Rôles et habilitations*

Implémentez la matrice T1 : création des rôles, alimentation, hiérarchie, attribution aux comptes, délégation au directeur. Vérifiez ensuite par une requête hiérarchique l'arbre des rôles de `DIR1`, puis par une requête qu'aucun utilisateur métier ne viole E1 et E2.

Exercice 6.5 — *T5 — Audit*

Mettez en place l'audit répondant aux exigences E6 à E8 :

- une politique unifiée `POL_BQ_CLIENTS` pour les modifications de fiches clients ;
- une politique `POL_BQ_SOLDES` qui ne trace que les **échecs** de modification directe des tables `COMPTES` et `OPERATIONS` ;
- une politique FGA `FGA_BQ_GROS_MONTANTS` sur les insertions dans `OPERATIONS` ;
- l'activation des politiques prédéfinies adaptées.

Pourquoi `POL_BQ_SOLDES` ne doit-elle pas tracer les succès ?

Exercice 6.6 — *T6 — Campagne de tests*

Exécutez les tests ci-dessous, complétez la colonne « Obtenu » et la référence de la trace d'audit lorsqu'il y en a une. Tout écart entre attendu et obtenu doit être expliqué et corrigé.

N°	Compte	Action	Attendu	Obtenu
1	GUI1	lire `v_clients_guichet`	succès	
2	GUI1	lire `revenu_mensuel` dans `clients`	ORA-00942	
3	GUI1	dépôt de 2 000 sur 1003 via `p_operation`	succès, `AGENT = GUI1`	
4	GUI1	`UPDATE comptes SET solde = 1e6`	ORA-01031 + trace `POL_BQ_SOLDES`	
5	GUI1	retrait de 60 000 sur 1002 via `p_operation`	succès + trace FGA	
6	GUI1	retrait de 5 000 sur 1005	ORA-20002	
7	GUI1	ouvrir une 2e session	ORA-02391	
8	CONS1	modifier le téléphone du client 2	succès + trace `POL_BQ_CLIENTS`	
9	CONS1	modifier le CIN du client 2	ORA-01031	
10	DIR1	accorder `R_BQ_GUICHET` à un nouveau compte `GUI3`	succès + trace `ORA_ACCOUNT_MGMT`	
11	DIR1	accorder `R_BQ_CONSEILLER` à `GUI3`	ORA-01932	
12	AUD1	lire `operations` puis la trace d'audit	succès	
13	AUD1	`DELETE FROM banque.operations`	ORA-01031	
14	GUI2	3 connexions avec un mauvais mot de passe, puis le bon	ORA-28000	
15	DEV_BQ	connexion `dev_bq[banque]`, index sur `operations(compte_id)`	succès	
16	APP_BATCH	dépôt d'intérêts de 1 200 sur 1002	succès	

Note : le compte `GUI3` (test 10) est créé au préalable par le DBA, sans aucun privilège, avec le profil `P_BQ_AGENT`.

Exercice 6.7 — *T7 — Revue des habilitations*

Produisez, à destination de l'auditeur, trois requêtes de revue :

1. pour chaque utilisateur métier : profil, état du compte, date d'expiration du mot de passe, rôles directs (une ligne par utilisateur, rôles concaténés avec `LISTAGG`) ;
2. pour chaque objet du schéma `BANQUE` : qui (rôle ou utilisateur) dispose de quel privilège ;
3. le nombre d'événements d'audit par politique et par utilisateur sur les 7 derniers jours.

6.4 Questions de synthèse

Exercice 6.8 — *À rédiger dans le rapport*

1. Le guichetier `GUI2` quitte la banque. Décrivez, commandes à l'appui, la procédure complète (accès, sessions, objets, audit). Faut-il supprimer le compte immédiatement ?
2. Pourquoi la procédure `P_OPERATION` protège-t-elle mieux les soldes qu'un `GRANT UPDATE (solde)` accordé aux guichetiers ?

3. Le directeur peut habiliter des guichetiers. Quel risque cette délégation introduit-elle et comment l'audit permet-il de le maîtriser ?
4. Le DBA (SYS) peut lire les revenus des clients. Est-ce acceptable ? Quelles solutions Oracle existent au-delà de ce TP (citez-les sans les mettre en œuvre) ?
5. Le conseiller voit *tous* les clients de l'agence. Comment limiter sa vue aux seuls clients dont il est le conseiller ?
6. Proposez deux améliorations de la politique de l'agence et justifiez-les.

6.5 Barème indicatif

Tâche	Critère principal	Points
T1 Matrice RBAC	cohérence avec E1–E3, héritages	**3**
T2 Profils	respect de E4–E5, justification	**2**
T3 Comptes	profils, mots de passe conformes, proxy	**2**
T4 Rôles	script complet, requêtes de contrôle vides	**3**
T5 Audit	quatre dispositifs opérationnels	**3**
T6 Tests	16 tests documentés, écarts expliqués	**4**
T7 Revue	trois requêtes exploitables	**1**
Questions de synthèse	qualité de l'argumentation	**2**
Total		**20**

A Annexes

A.1 Aide-mémoire : vues du dictionnaire

Vue	Contenu
`DBA_USERS`	comptes : état, profil, tablespaces, expiration, `COMMON`, `AUTHENTICATION_TYPE`
`DBA_TS_QUOTAS`	quotas par utilisateur et tablespace
`DBA_SYS_PRIVS`	privilèges système accordés (utilisateurs et rôles)
`DBA_TAB_PRIVS / DBA_COL_PRIVS`	privilèges objet, par objet / par colonne
`DBA_ROLES / DBA_ROLE_PRIVS`	rôles existants / rôles accordés
`ROLE_SYS_PRIVS, ROLE_TAB_PRIVS, ROLE_ROLE_PRIVS`	contenu des rôles
`SESSION_PRIVS / SESSION_ROLES`	privilèges et rôles actifs dans la session
`DBA_PROFILES`	paramètres de chaque profil
`PROXY_USERS`	autorisations de connexion proxy
`AUDIT_UNIFIED_POLICIES`	définition des politiques d'audit
`AUDIT_UNIFIED_ENABLED_POLICIES`	politiques actives et leur portée
`UNIFIED_AUDIT_TRAIL`	trace d'audit unifiée
`DBA_AUDIT_POLICIES / DBA_FGA_AUDIT_TRAIL`	politiques FGA / trace FGA (mode mixte)
`DBA_USED_* / DBA_UNUSED_*`	résultats de l'analyse des privilèges

A.2 Aide-mémoire : codes d'erreur du TP

Code	Signification	Chapitre
ORA-00942	table ou vue inexistante (ou invisible faute de privilège)	2, 3, 6
ORA-01017	nom d'utilisateur ou mot de passe invalide	1, 4, 5
ORA-01031	privilèges insuffisants	2, 3, 5, 6
ORA-01045	pas de `CREATE SESSION`	1
ORA-01536 / 01950	quota dépassé / aucun quota	1
ORA-01922	`CASCADE` requis pour `DROP USER`	1
ORA-01932	option `ADMIN` non accordée pour le rôle	6
ORA-01979	mot de passe de rôle absent ou invalide	3
ORA-02391 / 02396	trop de sessions / inactivité dépassée	4, 6
ORA-28000 / 28001 / 28002	compte verrouillé / mot de passe expiré / période de grâce	1, 4, 6
ORA-28003 / 28007 / 28221	complexité / réutilisation / `REPLACE` manquant	4
ORA-65094 / 65096	nom local / commun invalide	1

A.3 Script de nettoyage

[ADMIN] 99-nettoyage.sql

```
-- Audit
NOAUDIT POLICY pol_rh_salaires;  DROP AUDIT POLICY pol_rh_salaires;
NOAUDIT POLICY pol_bq_clients;   DROP AUDIT POLICY pol_bq_clients;
NOAUDIT POLICY pol_bq_soldes;    DROP AUDIT POLICY pol_bq_soldes;
NOAUDIT POLICY ora_account_mgmt;
EXEC DBMS_FGA.DROP_POLICY('RH','EMPLOYES','FGA_GROS_SALAIRES')
EXEC DBMS_FGA.DROP_POLICY('BANQUE','OPERATIONS','FGA_BQ_GROS_MONTANTS')
-- Utilisateurs (y compris les schémas applicatifs)
BEGIN
  FOR u IN (SELECT username FROM dba_users WHERE username IN
            ('DEV1','LECTEUR1','OMAR','A1','A2','LINA','SARA','AUDITEUR',

    'GUI1','GUI2','GUI3','CONS1','DIR1','AUD1','APP_BATCH','DEV_BQ','RH','BANQUE'))
  LOOP
    EXECUTE IMMEDIATE 'DROP USER ' || u.username || ' CASCADE';
  END LOOP;
END;
/
-- Rôles et profils
BEGIN
  FOR r IN (SELECT role FROM dba_roles WHERE role LIKE 'R\_RH\_%' ESCAPE '\'
            OR role LIKE 'R\_BQ\_%' ESCAPE '\' OR role = 'R_CONNEXION')
  LOOP
    EXECUTE IMMEDIATE 'DROP ROLE ' || r.role;
  END LOOP;
```

[ADMIN] 99-nettoyage.sql

```
END;
/
DROP PROFILE p_rh_standard CASCADE;  DROP PROFILE p_bq_agent CASCADE;
DROP PROFILE p_bq_conseil CASCADE;   DROP PROFILE p_bq_sensible CASCADE;
DROP PROFILE p_bq_service CASCADE;
```

✎ Attention !

Exécutez ce script uniquement en fin de TP, connecté en `SYS` dans `ORCLPDB`. L'utilisateur commun `C##SUPERVISEUR` se supprime depuis la racine : `DROP USER c##superviseur CASCADE;`.

A.4 Références

1. ORACLE. *Oracle Database Security Guide, 19c*. Chapitres « Managing Security for Oracle Database Users », « Configuring Privilege and Role Authorization », « Configuring Audit Policies ».
2. ORACLE. *Oracle Database SQL Language Reference, 19c*. `CREATE USER`, `GRANT`, `CREATE ROLE`, `CREATE PROFILE`, `CREATE AUDIT POLICY`.
3. ORACLE. *Oracle Database PL/SQL Packages and Types Reference, 19c*. `DBMS_FGA`, `DBMS_AUDIT_MGMT`, `DBMS_PRIVILEGE_CAPTURE`.
4. ORACLE. *Oracle Database Reference, 19c*. Vues `DBA_*` et `UNIFIED_AUDIT_TRAIL`.
5. ORACLE University. *Oracle Database 19c : Administration Workshop*. Leçons 19 à 22 (utilisateurs, privilèges et rôles, profils, audit).
6. BADIR, H. *Administration Oracle 19c — Chapitre 6 : Sécurité*. Support de cours, ENSA Tanger, 2026.

« La sécurité n'est pas un produit, c'est un processus. »
— Bruce Schneier